Data Structures

BS (CS) _Fall_2025

Lab_07 Manual

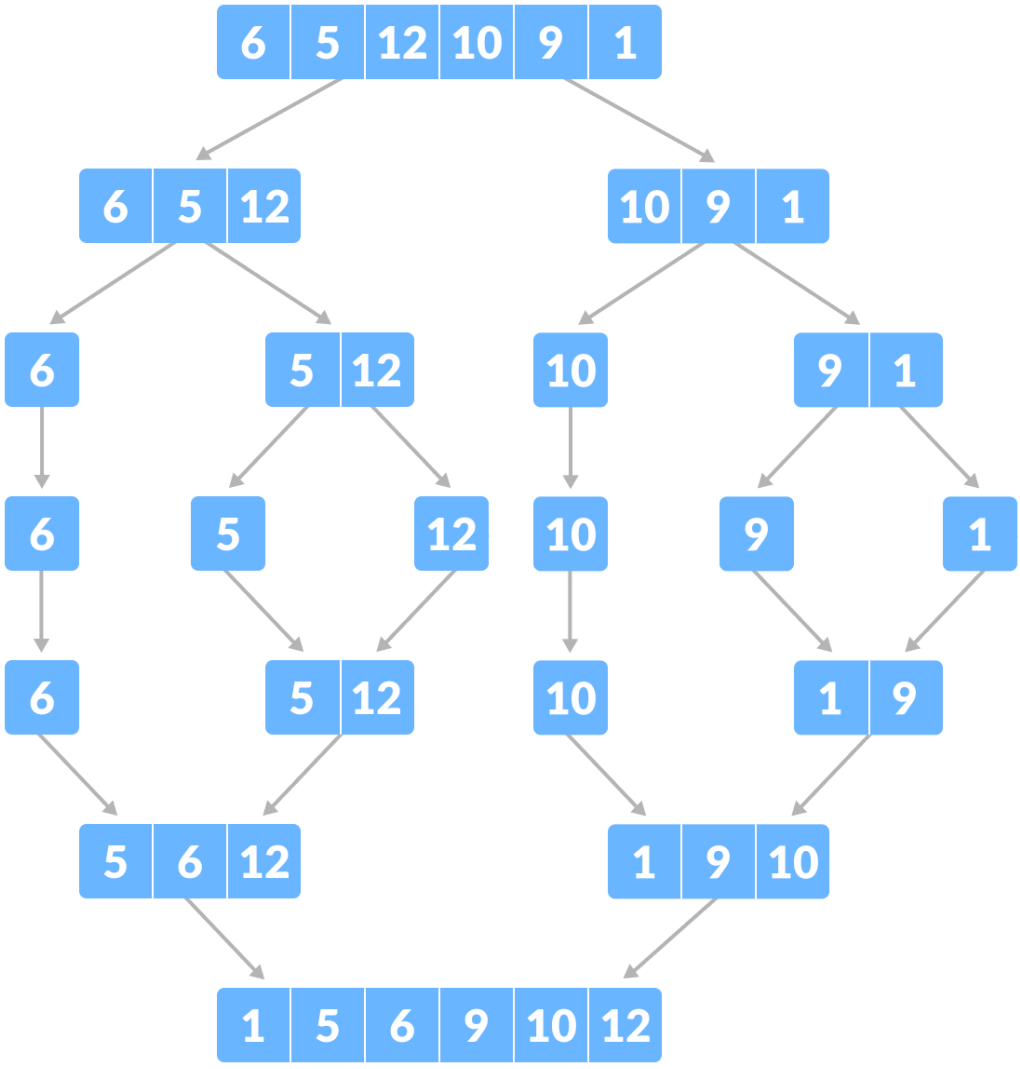
Learning Objectives:

1. Sorting using Arrays and Linked Lists

Merge Sort Algorithm:

Merge Sort is one of the most popular sorting algorithm that is based on the principle of Divide and Conquer algorithm.

Here, a problem is divided into multiple sub-problems. Each sub-problem is solved individually. Finally, sub-problems are combined to form the final solution.



As shown in the image below, the merge sort algorithm recursively divides the array into halves until we reach the base case of array with 1 element. After that, the merge function picks up the sorted sub-arrays and merges them to gradually sort the entire array.

Approach:

1. Divide: Divide the list or array recursively into two halves until it can no more be divided.
2. Conquer: Each subarray is sorted individually using the merge sort algorithm.
3. Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Example:
Let's look at the working of above example:

Divide:

- [38, 27, 43, 10] is divided into [38, 27] and [43, 10]
- [38, 27] is divided into [38] and [27]
- [43, 10] is divided into [43] and [10]

Conquer:

- [38] is already sorted.
- [27] is already sorted.
- [43] is already sorted.
- [10] is already sorted.

Merge:

- Merge [38] and [27] to get [27, 38]
- Merge [43] and [10] to get [10,43]
- Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]

Therefore, the sorted list is [10, 27, 38, 43]

Complexity Analysis of Merge Sort

Time Complexity:

- **Best Case:** O(n log n), When the array is already sorted or nearly sorted.
- **Average Case:** O(n log n), When the array is randomly ordered.
- **Worst Case:** O(n log n), When the array is sorted in reverse order.

Auxiliary Space: O(n), Additional space is required for the temporary array used during merging.

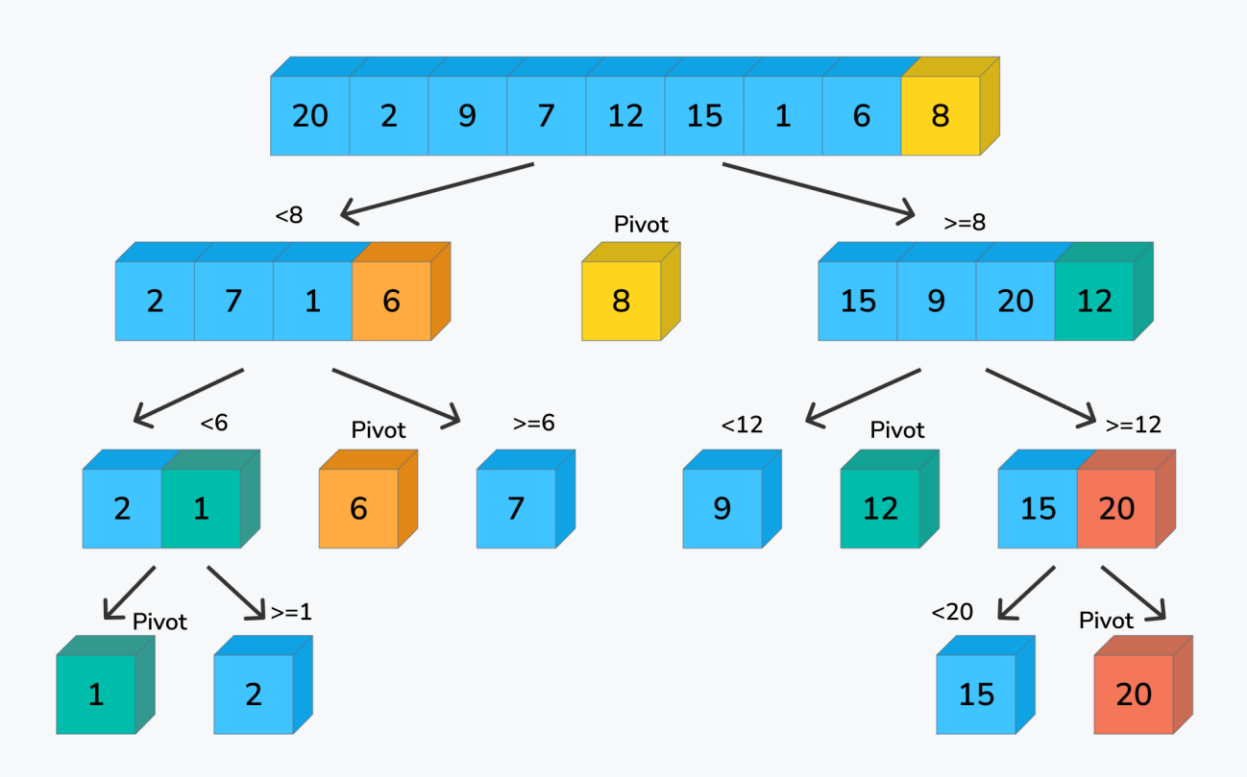
Quick Sort Algorithm:

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

It works on the principle of **divide and conquer**, breaking down the problem into smaller sub-problems.

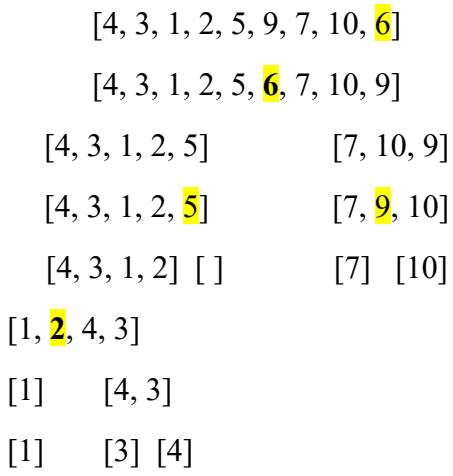
There are mainly three steps in the algorithm:

1. **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
2. **Partition the Array:** Re arrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then in its correct position, and we obtain the index of the pivot.
3. **Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays (left and right of the pivot).
4. **Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.



Example:

Forward Step:



Backward Step:

[1] 2 [3] [4] ➔ [1,2,3,4]

[1,2,3,4] 5 [] ➔ [1,2,3,4,5]

[1,2,3,4,5] 6 [7,10,9] ➔ [1,2,3,4,5,7,10,9]

[7] 9 [10] ➔ [7,9,10]

[1,2,3,4,5,7,10,9] 6 [7,9,10] ➔ [1,2,3,4,5,6,7,9,10]

Sorted Array ➔ [1,2,3,4,5,6,7,9,10]

Complexity Analysis of Quick Sort

Time Complexity:

- **Best Case:** ($\Omega(n \log n)$), Occurs when the pivot element divides the array into two equal halves.
- **Average Case** ($\theta(n \log n)$), On average, the pivot divides the array into two parts, but not necessarily equal.
- **Worst Case:** ($O(n^2)$), Occurs when the smallest or largest element is always chosen as the pivot (e.g., sorted arrays).